omnya lab



How to build a 3D with AI and animate it

▶ The method behind the tower video

Start here

what you'll have at the end

- ✓ The photo of your tower before it is built, from a plan and a block model.
- ✓ A site where the buyer picks the building, the floor and the apartment.
- ✓ The flight in through the glass and the walk from room to room, all real video.
- ✓ The way back to the tower in one move, with the logo at the end.
- ✓ One link for the developer, and it works on a phone.

What you need

Claude Code
It reads this PDF, writes the code and checks every step with you.

Higgsfield, connected
Turn on its connector in Claude so it can make the photos and the clips, or use the image and video tool you already have.

ffmpeg
To cut each clip on the exact frame, and Claude installs it for you.

A free Vercel account
To put the site online on a private link.

The fast way

give this PDF to Claude Code

Open Claude Code in an empty folder, drop this PDF in it and paste the prompt below: Claude reads the PDF page by page, asks you a few questions about your tower and then builds it with you step by step, showing you the result every time before it moves on.

```
I bought the Full Tower Blueprint by Omnya Lab. The PDF is in this folder:
read it page by page, from chapter 00 to chapter 12, before you do anything.

I want to build what you see in the Omnya Lab tower video
(instagram.com/omnyalab): a tower that looks built, and a site where a buyer
picks a floor, flies in through the glass and walks through the apartment.

First ask me what you need to know about my tower: plans, number of floors,
heights, where it stands, which apartment to show. Then follow the PDF in
order. After every step show me the result (the 3D, the photo, the clip, the
page in the browser) and wait for my OK before the next one. Use Higgsfield
through its connector for every photo and every video. If something fails one
of the checks in the PDF, fix it before you go on.
```

The Full Tower Blueprint is the complete method behind SAMAR Residences, every file, every command and every line of code we use. Chapter 00 gives you the tower photo with every prompt, word for word. From chapter 01 on you have the tower, and now you want a buyer to choose a building, a floor and an apartment, and to see that apartment from the inside before a single slab is poured. You will build it the way we built SAMAR, with real video, an SVG layer traced on the first frame and about three hundred lines of plain JavaScript. No game engine, no plugin, nothing to install on the buyer's phone.

You need the tower images from chapter 00, an account on Higgsfield (Seedance and Kling), ffmpeg, a code editor and a free Vercel account. Claude Code can write and fix most of the code with you, and we tell you where it helps the most.

CHAPTER 00

The tower photo every prompt, word for word

This is the part the free Tower Blueprint only shows from outside: how a few numbers become a block model, and how the block model becomes the photo of a tower that looks built.

The massing from the numbers

Ask Claude for the block model in the browser, with the real numbers of your tower. This is the prompt we used for two towers by the sea:

```
Build a 3D massing model in three.js, in real metres, on a plot 70 × 45 m
by the sea. A podium of 4 floors at 4.5 m. On it two residential towers of
42 and 55 floors, 3.4 m floor to floor, each one stepping back 3 m every 12
floors. Draw a thin line at every slab and balconies 2.5 m deep. Put the
neighbouring towers around as plain grey boxes at their real heights.
Camera on the beach 600 m away, at 1.7 m, 50 mm, vertical lines straight.
```

The height is the floors times 3.3 to 3.5 m, plus 4.5 to 6 m for each podium floor and 8 to 10 m for the crown. Get these right and the tower looks real before any material goes on.

Draw every floor, not just the box

The AI copies what it sees, so a plain box comes back as a generic glass tower. The 3D has to show what makes yours different:

- One line for every slab, so the floors can be counted.
- Balconies, setbacks and terraces at their real depth.
- The podium and the entrance, where people look first.
- The crown: the last floors, the roof terrace, the pool.
- The ground in front: sea, road or park, and the towers next door.

The camera, far and low

- Far away, 300 to 800 m, across the water or from the street.

- 50 to 85 mm. A wide lens up close makes the tower lean back.
- Vertical lines straight, like a shift lens: never tilt up.
- The tower fills two thirds of the height, with sky above the crown.

Name it in three parts

Leave one out and the model fills it with its favourite tower: blue glass, the same balcony on every floor, nothing on top. Name the crown (the last floors, lights, roof terrace), the body (glass tint, frames, balconies, fins) and the podium (stone, lobby glass, canopy, gardens), each with its material and colour.

The city around it

On Higgsfield, with Nano Banana Pro, upload the 3D screenshot first, then a real sample of the facade materials. Then say where the tower stands and what time it is:

```
Image 1 is the 3D: keep the camera, the two towers, the number of floors
and every setback exactly as they are. Floor-to-ceiling glass with a warm
bronze tint, slim champagne aluminium frames, balconies with clear glass
rails, as in image 2. A podium in honed travertine with a double-height
glass lobby. Behind, the Dubai Marina skyline, the grey boxes become the
real neighbouring towers. Sun low on the right at six in the evening.
Editorial architecture photograph, natural colour, fine film grain.
```

Words to drop, and what to write

Drop	Write
futuristic skyscraper	the floors, the height and the setbacks
iconic, luxury	the shape: two towers, a step every 12 floors
glass facade	the tint, the frame colour, the balcony depth
Dubai skyline	which district is behind, seen from where
dramatic dusk	where the sun is and what time it is

Never write the name of a real tower you like: the model copies it, logo and all.

Count the floors before you send it

If one check fails, don’t edit the picture: write the whole prompt again with the fix inside and generate from the 3D. Upscale only the image you keep.

- The same number of floors and setbacks as the 3D.
- Vertical lines straight, the tower not leaning back.
- Balconies that line up floor after floor, with no melted slabs.
- Cars and people at the right size next to the podium.
- At night, not every window lit, and no invented logos.

CHAPTER 01

What you are building and why it sells

A developer selling off plan has floor plans, a price list and a few renders. The buyer still has to imagine which floor is theirs and what they see from it. The experience answers that in under a minute, in this order:

1. **The site:** One or more towers seen from far away. The buyer clicks a building.
2. **The floor:** Moving over the tower lights up one floor at a time, with its number. A glass panel shows level, unit type, area and view.
3. **The apartment:** The buyer picks an apartment on the floor plan of that level, or goes straight in.
4. **The flight:** The camera flies from the tower to the terrace of that floor, through the glass and into the living room.
5. **The rooms:** Pills on the video lead to the next room (living room, then bathroom or bedroom).
6. **The return:** "Back to the tower" plays the whole way back in one move, and the project logo builds up on the last stretch.
7. **The action:** "Register interest" stays on screen the whole time.

Everything you see on screen is video. The only thing drawn by code is the light on the floors, the panel and the pills. That is why it looks like a film and still loads on a phone.

CHAPTER 02

The shot list before you generate anything

Write the whole path on paper first. Every arrow in the experience is one clip, and every clip has to start on the exact last frame of the one before. For one tower with one showcase apartment the list is short:

Clip	From	To	Tool
fly	the tower, far away	the terrace, then the living room	Seedance, start and end frame
enter	last frame of fly	inside the living room	Kling, start frame
bath	last frame of enter	the bathroom	Kling, start frame
back-*	any room	the tower	no generation: the same frames in reverse

Two rules decide whether the result looks real. All clips are exactly 1920 × 1080 at 24 frames per second, so they swap without a jump. And the tower frame you trace the floors on is the first frame of the flight, never a separate still, so the floors line up with the video to the pixel.

> One showcase apartment is enough to sell the whole tower. Buyers choose a floor for the view and the height; the interior shows the finish level. If the developer wants each unit type inside, repeat chapter 03 once per type and share the tower and the flight.

CHAPTER 03

The clips one start frame at a time

The flight

On Seedance give the tower image as start frame and your best terrace image of the same floor as end frame. Describe one continuous camera move: "slow forward drone flight toward the 15th floor terrace, the camera keeps rising slightly, steady, no cuts, no text". Generate a few and keep the one where the facade does not bend.

Extract the real last frame

Each next clip starts from the last frame of the one you keep. Be careful: `-sseof -0.04` in ffmpeg often gives you the second to last frame, and that tiny jump is what makes a cut visible. Count the frames and take the last one by number:

```
ffprobe -v error -count_frames -select_streams v:0 \
  -show_entries stream=nb_read_frames -of csv=p=0 fly.mp4
# say it prints 120: the last frame is number 119
ffmpeg -i fly.mp4 -vf "select='eq(n\,119)'" -vframes 1 fly_last.png
```

On SAMAR we cut the flight at frame 119 because that is where the living room is fully in view. Pick your own cut point, then extract that exact frame.

The rooms

On Kling, start frame = `fly_last.png`, prompt = one slow move to the next room ("the camera walks slowly from the living room toward the bathroom door and enters, steady, no cuts"). Repeat for each room, always starting from the real last frame of the clip before.

Match the colour

Two generators never give you the same colour. If one room looks colder, make a LUT once (in DaVinci Resolve or Photoshop, matching it to the previous clip) and bake it in:

```
ffmpeg -i bath_raw.mp4 -vf "lut3d=file=bath_match.cube,scale=1920:1080" \
  -c:v libx264 -preset slow -crf 10 -pix_fmt yuv420p -movflags +faststart
bath.mp4
```

> What not to do, because we tried it: optical-flow interpolation between clips, crossfades, motion blur on the joins, stills between two videos. All of them look cheap next to the original footage. Keep only real frames.

CHAPTER 04

The way back without generating anything

The return to the tower is the same footage played backwards, joined into one clip per room, so the camera never stops on the way out. From the bathroom it is: bathroom reversed, then living room reversed, then flight reversed.

```
# from the terrace: flight frames 0..119, reversed
ffmpeg -i fly.mp4 -filter_complex \
 "[0:v]select='lte(n\,119)',setpts=N/24/TB,reverse[o]" -map "[o]" \
 -c:v libx264 -preset slow -crf 10 -pix_fmt yuv420p -movflags +faststart
back-terrace.mp4

# from the bathroom: three clips reversed and joined, the shared frames
kept once
ffmpeg -i fly.mp4 -i enter.mp4 -i bath.mp4 -filter_complex \
 "[2:v]setpts=N/24/TB,reverse[b];\
  [1:v]select='lte(n\,95)',setpts=N/24/TB,reverse[l];\
  [0:v]select='lte(n\,118)',setpts=N/24/TB,reverse[f];\
  [b][l][f]concat=n=3:v=1:a=0[o]" -map "[o]" \
 -c:v libx264 -preset slow -crf 10 -pix_fmt yuv420p -movflags +faststart
back-bath.mp4
```

Notice `lte(n\,118)` in the last line: frame 119 is already the first frame of the living room clip, so we keep it once. If you keep it twice the camera stutters on the join.

Even out the speed

AI clips slow down at the start and at the end. Reversed and joined, those slow stretches become little stops. This script measures how much each frame moves and chooses which original frames to show, so the camera keeps one speed and eases in on the tower. It makes no new pixels.

```
# even_return.py  src.mp4  out.mp4  seconds      (pip install numpy opencv-
python)
import sys, subprocess, numpy as np, cv2
src, out, dur = sys.argv[1], sys.argv[2], float(sys.argv[3]); FPS = 24
cap = cv2.VideoCapture(src); small = []
while True:
    ok, f = cap.read()
    if not ok: break
    small.append(cv2.resize(cv2.cvtColor(f, cv2.COLOR_BGR2GRAY), (320,
180)).astype(np.float32))
d = np.array([np.abs(small[i] - small[i-1]).mean() for i in range(1,
len(small))])
d = np.maximum(d, 0.05 * np.median(d))
C = np.concatenate([[0], np.cumsum(d)]); C /= C[-1]
ease = lambda t: t * t * (3 - 2 * t)
n = int(round(dur * FPS))
pick = [int(np.clip(np.searchsorted(C, ease(k / (n - 1))), 0, len(small) -
1)) for k in range(n)]
p = subprocess.Popen(['ffmpeg','-v','error','-y','-f','rawvideo','-
pix_fmt','bgr24','-s','1920x1080',
  '-r',str(FPS),'-i','-','-c:v','libx264','-preset','slow','-crf','10','-
pix_fmt','yuv420p',
  '-movflags','+faststart',out], stdin=subprocess.PIPE)
cap = cv2.VideoCapture(src); i = -1; frame = None
for k in pick:
    while i < k:
        ok, frame = cap.read(); i += 1
    p.stdin.write(frame.tobytes())
p.stdin.close(); p.wait()
```

Run it on each `back-*.mp4` and try a few durations. The right one is the shortest where nothing jumps.

CHAPTER 05

Tracing the floors on the first frame

Export frame 0 of `fly.mp4` as a PNG. Every floor becomes one closed shape on that image, in its own pixel space (0 to 1920, 0 to 1080). There are three ways to get the shapes, from fastest to most precise:

1. **By hand in Figma or Illustrator:** Put the PNG on a 1920 × 1080 artboard, draw one polygon per floor band, export as SVG and copy the `d` of each path.
2. **From the slab lines:** If your tower has visible slab edges, trace only the top and bottom edge of the facade as two polylines and split the height evenly into floors. This is what we did on SAMAR: points every 6 pixels along each slab edge, so the band follows the curve of the facade.
3. **From the 3D model:** If you still have the block model with the same camera, render one mask per floor (each floor a flat white material, everything else black) and let Claude Code turn the masks into paths with OpenCV `findContours`.

Store each floor as data, bottom to top. The numbers are the ones the sales team gives you:

```
const floors = [
  { n: 1,  d: "M566 767.9 L572 768.8 ... Z", xr: 878, mid: 790,
    unit: "Two bedrooms", area: 142, out: "Marina view" },
  // ...
  { n: 15, d: "M... Z", xr: 902, mid: 512,
    unit: "Three bedrooms + study", area: 238, out: "Sea and marina" },
];
```

`xr` is the right edge of the band and `mid` its vertical centre: the floor number sits there, on a short line, outside the tower.

Draw them over the video

The trick that keeps the floors on the tower at every screen size is to give the SVG the same box and the same crop as the video: `viewBox="0 0 1920 1080"` with `preserveAspectRatio="xMidYMid slice"` is exactly what `object-fit: cover` does to the video.

```
<video class="clip on" id="v-fly" muted playsinline preload="auto"></video>
<svg class="floors" id="floors" viewBox="0 0 1920 1080"
preserveAspectRatio="xMidYMid slice">
  <defs>
    <filter id="soft"><feGaussianBlur stdDeviation="7"/></filter>
    <linearGradient id="fade" x1="0" x2="1">
      <stop offset="0" stop-color="#ffbf55" stop-opacity="0"/>
      <stop offset=".5" stop-color="#ffcf8a" stop-opacity=".9"/>
      <stop offset="1" stop-color="#ffbf55" stop-opacity="0"/>
    </linearGradient>
  </defs>
</svg>
```

```
.clip { position:absolute; inset:0; width:100%; height:100%; object-
fit:cover; opacity:0; }
.clip.on { opacity:1; }
svg.floors { position:absolute; inset:0; width:100%; height:100%; }
.floor path.hit { fill: transparent; cursor: pointer; }
.floor path.shape { fill:url(#fade); opacity:0; mix-blend-mode:screen;
filter:url(#soft);
                    transition: opacity .3s; pointer-events:none; }
.floor.hover path.shape { opacity:.9; }
.floor line, .floor text { opacity:0; transition: opacity .25s; pointer-
events:none; }
.floor.hover line, .floor.hover text { opacity:1; }
```

The warm light is a blurred gradient in screen blend mode, so it reads as light coming from inside the glass and not as a coloured sticker. Each floor gets two copies of the same path: an invisible one that catches the mouse and the visible one that glows.

CHAPTER 06

The floor picker panel, hover and click

```
<aside class="panel glass" id="panel">
  <h1>Choose<br>your floor</h1>
  <div class="level"><small>Level</small><span class="num"
id="num">15</span></div>
  <div class="unit" id="unit"></div>
  <div class="meta" id="meta"></div>
  <button class="enter pill btn-blue" id="enter">Enter level 15</button>
</aside>
```

```
const NS = 'http://www.w3.org/2000/svg', svg =
document.getElementById('floors');
const $ = id => document.getElementById(id);
let current = null;

floors.forEach(f => {
  const g = document.createElementNS(NS, 'g'); g.setAttribute('class',
'floor');
  g.innerHTML = `<path class="hit" d="${f.d}"/><path class="shape"
d="${f.d}"/>
    <line x1="${f.xr+26}" x2="${f.xr+76}" y1="${f.mid}" y2="${f.mid}"/>
    <text x="${f.xr+88}" y="${f.mid+8}">${f.n}</text>`;
  g.addEventListener('mouseenter', () => pick(f, g));
  g.addEventListener('click', () => { pick(f, g); fly(); });
  f.el = g; svg.appendChild(g);
});

function pick(f) {
  floors.forEach(x => x.el.classList.toggle('hover', x === f));
  if (f === current) return;
  current = f;
  gsap.fromTo('#num', { yPercent: 30, opacity: 0 }, { yPercent: 0, opacity:
1, duration: .35, ease: 'power2.out' });
  $('num').textContent = f.n;
  $('unit').textContent = f.unit;
  $('meta').textContent = `${f.area} m² · ${f.out}`;
  $('enter').textContent = `Enter level ${f.n}`;
}
pick(floors[14]);   // open on the showcase floor
```

The glass

Clear glass with a heavy blur and a thin bright rim, no coloured fill. Solid blue for the one action that goes forward, white for everything else.

```
.glass { background: rgba(255,255,255,.10);
  backdrop-filter: blur(28px) saturate(180%); -webkit-backdrop-filter:
blur(28px) saturate(180%);
  box-shadow: inset 0 0 0 .5px rgba(255,255,255,.45), inset 0 1px 1px
rgba(255,255,255,.35),
              0 8px 30px rgba(0,0,0,.18); }
.btn-blue  { background:#0a84ff; color:#fff; }
.btn-white { background:#fff; color:#1d1d1f; }
.pill { height:40px; padding:0 18px; border-radius:999px; font-weight:600;
}
```

Use GSAP from cdnjs for the few animations (`gsap@3.12.5`). Every floor stays selectable, even the ones you have no interior for: the flight is the same, and the panel is what changes.

CHAPTER 07

The building and the apartment two more layers on the same idea

More than one building

A masterplan with several towers works exactly like the floors, one level up. Generate one wide aerial image of the whole site, trace one shape per building, and on click play a short Seedance move from the aerial to that tower's first frame (start frame = aerial, end frame = frame 0 of that tower's `fly.mp4`). Then the floor picker of chapter 06 opens on that tower.

```
const buildings = [
  { id: 'a', name: 'Tower A', d: 'M... Z', intro: 'assets/site-to-a.mp4',
floors: floorsA },
  { id: 'b', name: 'Tower B', d: 'M... Z', intro: 'assets/site-to-b.mp4',
floors: floorsB },
];
```

The apartment on the floor

When a floor has more than one unit, the buyer picks the unit on the plan of that level before flying in. Clean the architect's plan into a flat drawing (white walls on a dark background works well over video), and draw one shape per unit in the same way you drew the floors, this time in the plan's own pixel space.

```
const units = {
  15: [
    { id: '15A', d: 'M120 80 L540 80 L540 420 L120 420 Z', type: 'Three
bedrooms + study',
      area: 238, status: 'available', interior: 'type-3b' },
    { id: '15B', d: 'M540 80 L900 80 L900 420 L540 420 Z', type: 'Two
bedrooms',
      area: 164, status: 'reserved', interior: 'type-2b' },
  ],
};

function showPlan(level) {
  const list = units[level.n];
  if (!list) return fly();                          // one unit per floor:
go straight in
  plan.innerHTML = list.map(u =>
    `<path class="unit ${u.status}" data-id="${u.id}"
d="${u.d}"/>`).join('');
  plan.querySelectorAll('.unit.available').forEach(p =>
    p.addEventListener('click', () => { chosen = list.find(u => u.id ===
p.dataset.id); fly(); }));
  gsap.to('#planbox', { opacity: 1, duration: .4 });
}
```

Reserved and sold units stay on the plan in grey and are not clickable: the buyer sees that the building is selling. `interior` says which set of room clips to play, so ten units of the same type share one set of videos.

> Keep the availability in one JSON file (`units.json`) that the sales team can edit. The page reads it on load, so marking a unit as sold never needs a new deploy of the code.

CHAPTER 08

The player so nothing stalls

One `<video>` element per clip, stacked, all muted and `playsinline`. Each file is downloaded whole into memory before the experience starts, because a video that buffers halfway through a flight breaks the illusion at once.

```
const CLIPS = { fly: 'assets/fly.mp4', enter: 'assets/enter.mp4', bath:
'assets/bath.mp4',
  'back-terrace': 'assets/back-terrace.mp4', 'back-living': 'assets/back-
living.mp4',
  'back-bath': 'assets/back-bath.mp4' };
let ready = 0; const total = Object.keys(CLIPS).length;
Object.entries(CLIPS).forEach(async ([id, src]) => {
  const blob = await (await fetch(src)).blob();
  const v = $('v-' + id);
  v.src = URL.createObjectURL(blob);
  v.addEventListener('loadeddata', () => {
    ready++; $('loader').textContent = `Loading ${Math.round(ready / total
* 100)}%`;
    if (ready === total) gsap.to('#loader', { opacity: 0 });
  }, { once: true });
  v.load();
});
```

Swapping clips is a show then hide, two animation frames apart, so there is never a black frame between them:

```
let showing = $('v-fly');
function swapTo(el) {
  el.classList.add('on');
  if (showing && showing !== el) {
    const old = showing;
    requestAnimationFrame(() => requestAnimationFrame(() =>
old.classList.remove('on')));
  }
  showing = el;
}
function play(id, onStart) {
  const v = $('v-' + id);
  return new Promise(resolve => {
    v.pause(); v.currentTime = 0;
    const start = () => { swapTo(v); v.play(); onStart && onStart(v);
      v.addEventListener('ended', resolve, { once: true }); };
    if (v.readyState >= 2 && !v.seeking) start(); else
v.addEventListener('seeked', start, { once: true });
  });
}
```

Weight

We encode at `-crf 10` because the master has to survive a 21:9 monitor. For the public page make a second set at `-crf 20` and a 1280 × 720 set for phones, and choose by screen width before the fetch. Test the total on a real 4G connection: the loader must reach 100 % in a few seconds.

CHAPTER 09

Rooms and hotspots a small state machine

Room pills sit on the footage at a fixed point of the frame (`data-u` , `data-v` from 0 to 1). This function moves them with the video's crop at any window size:

```
function placeSpots() {
  const iw = 1920, ih = 1080, s = Math.max(innerWidth / iw, innerHeight /
ih);
  const ox = (innerWidth - iw * s) / 2, oy = (innerHeight - ih * s) / 2;
  document.querySelectorAll('.spot').forEach(el => {
    el.style.left = (ox + el.dataset.u * iw * s) + 'px';
    el.style.top  = (oy + el.dataset.v * ih * s) + 'px';
  });
}
placeSpots(); addEventListener('resize', placeSpots);
```

The experience is always in one state, and one flag blocks double clicks while a clip plays. This is the whole logic:

```
let state = 'tower', busy = false;
async function fly() {
  if (busy || state !== 'tower' || ready < total) return;
  busy = true;
  gsap.to(['#panel', '#floors'], { opacity: 0, duration: .4 });
  $('back').classList.add('on');
  document.querySelectorAll('.lvl').forEach(el => el.textContent = `Level
${current.n}`);
  await play('fly');
  state = 'terrace'; busy = false; captions('cap-terrace'); $('spot-
in').classList.add('on');
}
$('spot-in').addEventListener('click', async () => {
  if (busy || state !== 'terrace') return;
  busy = true; captions(); await play('enter');
  state = 'living'; busy = false; captions('cap-living'); $('spot-
bath').classList.add('on');
});
$('spot-bath').addEventListener('click', async () => {
  if (busy || state !== 'living') return;
  busy = true; captions(); await play('bath');
  state = 'bath'; busy = false; captions('cap-bath');
});
```

The caption in the corner says where you are ("Level 15 · Living Room") and changes only when the camera has stopped.

CHAPTER 10

The return and the ending the moment people remember

```
$('back').addEventListener('click', async () => {
  if (busy || !['terrace', 'living', 'bath'].includes(state)) return;
  busy = true; captions();
  const id = { terrace: 'back-terrace', living: 'back-living', bath: 'back-
bath' }[state];
  await play(id, v => { const lead = v.duration * .55;
    gsap.delayedCall(v.duration - lead, () => finale(lead)); });
  state = 'end'; busy = false; $('back').classList.remove('on');
$('restart').classList.add('on');
});
function finale(dur) {
  return gsap.timeline()
    .fromTo('#finale-logo', { opacity: 0, y: 30, scale: .94, filter:
'blur(14px)' },
            { opacity: 1, y: 0, scale: 1, filter: 'blur(0px)', duration:
dur, ease: 'power2.out' })
    .fromTo('#finale-sub', { opacity: 0, y: 10 }, { opacity: 1, y: 0,
duration: dur * .6 }, dur * .4);
}
```

The logo starts building up while the camera is still pulling back, so the last second of the return and the title are one move. "Start again" fades the title out and brings the floor picker back on the same frame, with no reload.

CHAPTER 11

Phones where most buyers will open it

- The panel moves to the bottom of the screen, 16 px from each side, and the floors keep the full height above it.
- There is no hover on a phone: the first tap on a floor selects it and lights it up, the "Enter" button flies in. Never fly on the first tap.
- For a vertical screen, generate a vertical version of the tower frame and of the flight (9:16) and trace the floors again. Cropping the horizontal video to a phone loses half the tower.
- Every pill is at least 44 × 44 points, the minimum Apple uses for anything you touch.
- All videos `muted playsinline`, or iPhone opens them full screen and the experience is gone.

```
@media (max-width: 760px) {
  .panel { top:auto; bottom:16px; right:16px; left:16px; width:auto;
transform:none; padding:22px; }
  .panel h1 { font-size: 30px; }
  .level .num { font-size: 52px; }
}
```

CHAPTER 12

Online and delivered and what to charge for it

Publish

The whole experience is one `index.html` and an `assets` folder, so any static host works. On Vercel: drag the folder in, or push it to a GitHub repository and import it. Before the launch, send the developer a private link with `<meta name="robots" content="noindex">`, and remove it when they go public.

Connect it to sales

"Register interest" opens a short form with the chosen building, floor and unit already filled in, so the sales team knows what the buyer looked at before calling. Pass them in the link:

```
$('register').addEventListener('click', () => {
  const q = new URLSearchParams({ tower: building?.id || 'a', level:
current.n, unit: chosen?.id || '' });
  location.href = '/register?' + q;     // or the developer's CRM form
});
```

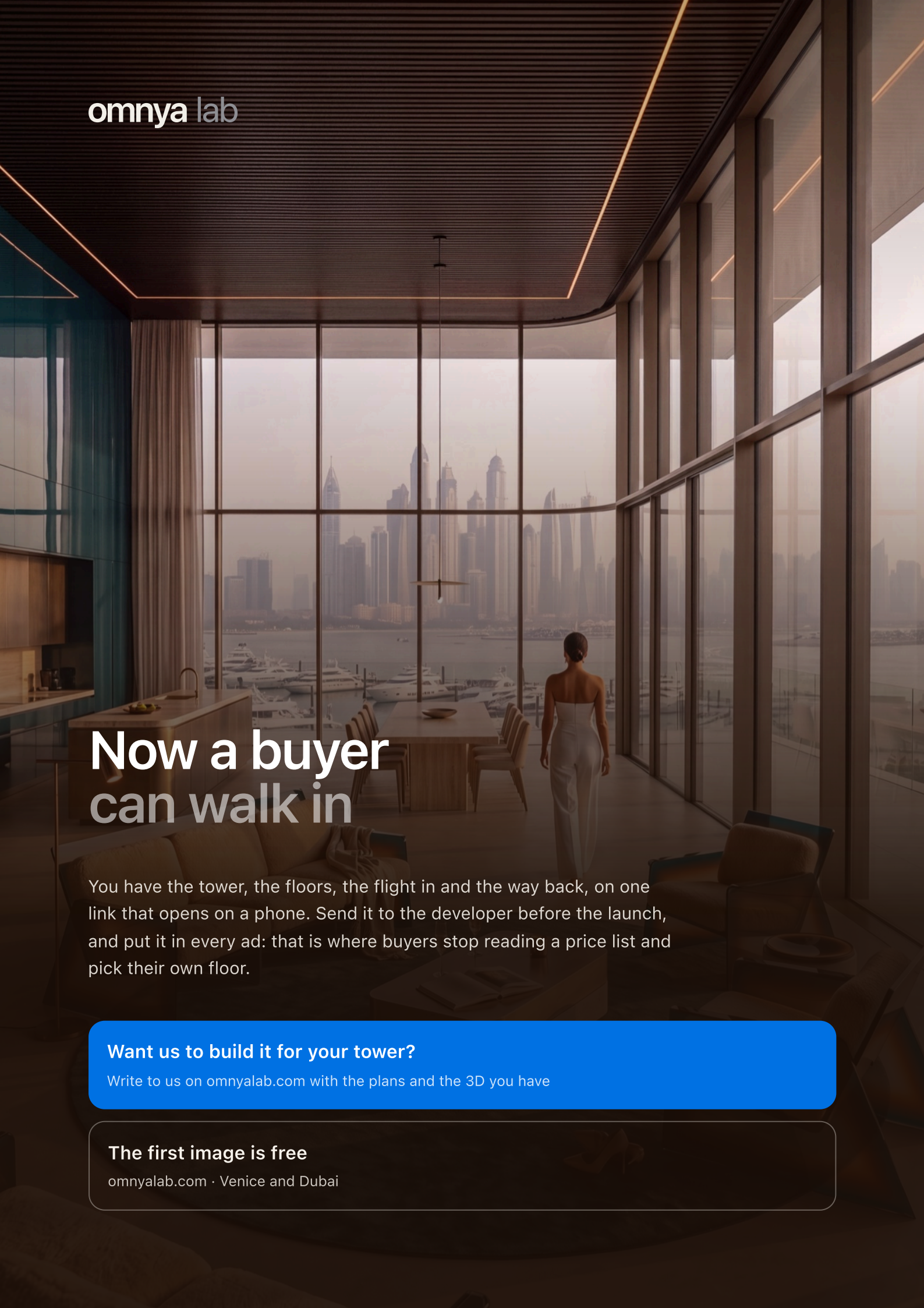
The check before you send it

1. Every join plays without a jump, a flash or a stop, in both directions.
2. Every floor band sits on its floor on a 13" laptop, a 21:9 monitor and a phone.
3. Floor numbers, unit types, areas and views match the developer's price list, one by one.
4. Double clicks during a flight do nothing.
5. The loader reaches 100 % on 4G, and nothing plays before it does.
6. No text, logo or watermark in the videos that the developer did not give you.

What you are selling

The developer is not buying a website. They are buying the minute in which a buyer stops reading a price list and picks their own floor, and that minute happens in the sales office, on the broker's phone and in every ad that points to the link. Price it as part of the launch, next to the film and the renders, because it uses the same images.

If you want us to build it for one of your towers, write to us from omnyalab.com and send the floor plans and the 3D you have.



omnya lab

Now a buyer can walk in

You have the tower, the floors, the flight in and the way back, on one link that opens on a phone. Send it to the developer before the launch, and put it in every ad: that is where buyers stop reading a price list and pick their own floor.

Want us to build it for your tower?

Write to us on omnyalab.com with the plans and the 3D you have

The first image is free

omnyalab.com · Venice and Dubai